Movie Ticketing System

Software Requirements Specification

Version 1

February 12, 2026

Group 14
Yosef Pilip, Ian Gillespie, Jonathan Clemente

Prepared for
CS 250 - Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2026

Revision History

Date	Description	Author	Comments
2/19/26	Version 1	Group 14	First Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Jonathan Clemente	Software Eng.	
	Ian Gillespie	Software Eng.	
	Yosef Pilip	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces 3
3.1.2 Hardware Interfaces 3
3.1.3 Software Interfaces 3
3.1.4 Communications Interfaces 3
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> 3
3.2.2 <Functional Requirement or Feature #2> 3
3.2.3 <Functional Requirement or Feature #3> 3
3.2.4 <Functional Requirement or Feature #4> 3
3.3 Use Cases 3
3.3.1 Use Case #1 3
3.3.2 Use Case #2 3
3.3.3 Use Case #3 3
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> 3
3.4.2 <Class / Object #2> 3
3.4.3 <Class / Object #3> 3
3.5 Non-Functional Requirements 4
3.5.1 Performance 4
3.5.2 Reliability 4
3.5.3 Availability 4
3.5.4 Security 4
3.5.5 Maintainability 4
3.5.6 Portability 4
3.6 Inverse Requirements 4
3.7 Design Constraints 4
3.8 Logical Database Requirements 4
3.9 Other Requirements 4

4. Analysis Models 4

4.1 Sequence Diagrams 5
4.3 Data Flow Diagrams (DFD) 5
4.2 State-Transition Diagrams (STD) 5

5. Change Management Process 5

A. APPENDICES.....5

A.1 APPENDIX 1.....5
A.2 APPENDIX 2.....5

1. Introduction

This Software Requirement Specification document is used to outline, define, and give requirements for a system's functions, purpose, use-cases, scope, characteristics, and constraints. In addition, this document will outline any interfacing requirements and models that may be applicable to this system.

1.1 Purpose

The purpose of this document is to outline a system and define/describe its requirements, constraints, and attributes for a group of theater employees, specifically the software development team and upper management for the company.

1.2 Scope

(1) This program is a web-based Movie Ticketing System that will work for a theatre company for theaters in the relevant areas and will be able to handle transactions and give users options in terms of language, ticketing, location, showtime, seat choice, and ticket delivery. Tickets themselves will also be unique and non-replicable to prevent unauthorized use. The system will also allow for the purchase of tickets and concessions ahead of time.

It will also allow for a max number of tickets purchasable in one transaction, as well as applicable discounts (student, veteran, etc.) It will also include a queuing system for high-traffic showtimes. The system allows for protected administrative access from authorized users and employees in order to add or remove movie listings, update showtimes, display dates, display locations, display pricing, and notify the administrator if information is missing.

(2) The movie ticketing system will allow the customers to browse show times for movies and the locations supported. It will also allow customers to choose from available seats on a seat map as well as purchase the tickets. It will also allow for the ability to access purchased tickets for admission into the theatre (as well as transfer to mobile wallets).

It will also allow for email confirmation and user account logins for convenience purposes, as well as refunds, payment, and personal information, loyalty points, purchase history, and customer support. The ticketing system will also allow administrators to perform general maintenance on the user interface, as well as perform duties as mentioned in section 1.2.1 of the Scope. The system will be able to support a certain number of concurrent users.

(3) The first goal is to make the interfacing experience of choosing, purchasing, processing, accessing, browsing, and using digital movie tickets as seamless, convenient, efficient, and as simple as possible for customers. The next goal is to make sure the processing power of all relevant functions, as well as interfacing with external programs, such as mobile wallets, payment portals, and banks, is as streamlined and error-free as possible. The final objective is

to allow for robust, simple, readable, and efficient back-end management and processing of information for the administrative side of the system.

1.3 Definitions, Acronyms, and Abbreviations

Loyalty Account	An account that stores personal information, purchase history, and loyalty points for the movie theaters
Administrator	An authorized employee who has access to system data and can edit the features of the system
QR Code	A ticket in the form of a machine-readable code generated by the system, to be used at the theater

1.4 References

Unknown. (n.d.-a). *Theater Ticket System Q's*. Sign in to your account. https://sdsu.instructure.com/courses/197506/files/19593984?module_item_id=5531939

Unknown. (n.d.-b). *Theater Ticket System Requirements*. Sign in to your account. https://sdsu.instructure.com/courses/197506/files/19593986?module_item_id=5531940

Unknown. (n.d.-c). *Theather-ticketing-questions*. Sign in to your account. https://sdsu.instructure.com/courses/197506/files/19593990?module_item_id=5531941

Unknown. (n.d.-d). *Theather-ticketing-reqs*. Sign in to your account. https://sdsu.instructure.com/courses/197506/files/19593991?module_item_id=5531942

1.5 Overview

The rest of the Software Requirement Specification is organized into 5 parts, with appropriate subsections that include a general description with assumptions, constraints, and functions, a requirements section with all requirements for the system, including interfacing and use cases, a section with models showing a visual representation of the system, a change management section, and any applicable appendices.

2. General Description

This section provides a general overview of the Movie Ticketing System. It does not include detailed requirements but instead describes the overall context of the system, including its operating environment, intended users, constraints, and assumptions that may affect its requirements.

2.1 Product Perspective

The Movie Ticketing System is a web-based application developed for a single movie theater company. It serves as the primary online platform for ticket sales. The system operates entirely within the web browser and does not exist as a standalone mobile or desktop application.

The system integrates with external services, including an external payment service, currency conversion services, an email service for purchase confirmations, and review sources. It incorporates a secure ticket generation mechanism to ensure ticket authenticity and system reliability.

2.2 Product Functions

The system provides functionality for users to browse available movies, view showtimes, seating availability, and pricing across all theater locations. It allows users to select their seats, purchase tickets, and order food for pickup at the selected theater. It supports loyalty account registration and administrative management of theater operations. It also processes payments through an external payment service and generates confirmation emails for completed transactions.

The system provides mechanisms for secure transactions and fair services during high demand situations. It maintains purchase records and system logs for administrative use.

2.3 User Characteristics

The system will be suitable for customers who are teens or older. Users are expected to have basic familiarity with using web applications, but are not required to have any technical training. The system is intended for occasional use to purchase movie tickets and related services. Customers with and without loyalty accounts may access the system through web browsers on desktop or mobile devices. Administrators may use the system to access purchase records and system logs.

2.4 General Constraints

The system must operate as a web application that is accessible through standard web browsers and requires internet connectivity to function. The system supports multiple languages. The system must integrate with an external payment service that complies with relevant payment security standards. The system maintains pricing consistency and can operate under high traffic conditions.

2.5 Assumptions and Dependencies

It is assumed that the user has reliable internet access and will access the system using modern web browsers. It is assumed that the external payment service is operational. The system depends on the theater management system having accurate movie and showtime data. The system depends on the availability of an external email service for sending confirmations to users.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The user interface will use a GUI that is accessible through web browsers like chrome and firefox. Information about movie listings, times, locations, and seats will be presented on pages. The user input is given through buttons and text fields.

3.1.2 Hardware Interfaces

Purchased tickets are scanned through a QR-code scanner at the theatre. Tickets can also be bought at the theatre through touch-screen kiosks. Tickets can also be bought on Android and IOS mobile devices.

3.1.3 Software Interfaces

The system will use a payment processor such as PayPal to validate ticket purchases made by customers and allow for refunds. Email services like Gmail and Outlook will integrate with the system to allow customers to receive an email confirming their purchase and provide them with a QR-code to scan at the theatre.

3.1.4 Communications Interfaces

Secure HTTPS connections will allow the system to share and communicate with external services. Web browser access will be supported by Internet-based communication. Ticket scanners and kiosks will communicate with the backend through the theatre's secured Wi-Fi or by LAN.

3.2 Functional Requirements

3.2.1 <Functional Requirement or Feature #1>

3.2.1.1 Introduction

Customers should be able to create an account on the website and choose a language (English, Spanish, French). The account should be able to receive loyalty benefits and account history like purchases should be accessible.

3.2.1.2 Inputs

The customer enters an email address and creates a password.

3.2.1.3 Processing

The account is created and the account info is saved.

3.2.1.4 Outputs

The customer has an account, they can choose a language, and they can earn loyalty benefits. They can also view their past purchases and request refunds.

3.2.1.5 Error Handling

The customer will be notified if they input an invalid email address.

3.2.2 <Functional Requirement or Feature #2>

3.2.2.1 Introduction

The system should allow customers to search for a movie to see information and availability for watching the movie.

3.2.2.2 Inputs

The customer searches for the name of a movie, or the user browses for a movie.

3.2.2.3 Processing

The system gathers a collection of information related to the movie being searched for.

3.2.2.4 Outputs

The movie shows up with a description of the movie and an option to see the available locations, times, and seats for purchase.

3.2.2.5 Error Handling

If the movie being searched is not an available movie in the theatres, an error message will appear, notifying the user that the movie is not available. If certain desired showtimes are not available, the customer will be put in a queue if they wish to book that time.

3.2.3 <Functional Requirement or Feature #3>

3.2.3.1 Introduction

The system should allow customers to purchase movie tickets with optional concessions and receive a QR-code for scanning.

3.2.3.2 Inputs

The user clicks the purchase ticket button with the option of purchasing concessions.

3.2.3.3 Processing

The system registers that the payment button was clicked, the request for concessions is checked, the customer's payment information is registered and the payment is completed.

3.2.3.4 Outputs

The customer receives an email confirmation containing a QR-code to be scanned at the theatre.

3.2.3.5 Error Handling

The customer will be notified if they enter invalid card information or if their payment method did not register,

3.2.4 <Functional Requirement or Feature #4>

3.2.4.1 Introduction:

The administrator should be able to update and edit the website information in order to display relevant information.

3.2.4.2 Inputs:

The administrator enters editing mode and enters new text in input boxes.

3.2.4.3 Processing

The website recognizes the changes being made in editing mode.

3.2.4.4 Outputs

Website information is updated to display what the administrator changed or updated.

3.2.4.5 Error Handling

If certain important information is missing or not updated, the administrator will be notified.

3.3 Use Cases

3.3.1 Use Case #1

Name: Create an account and choose language
Description: Customers can create an account and choose a language.
Actors: Customers
Basic flow: Customers use their email and come up with a password to create an account on the website.
Customers login to the website with their account.
Customers choose a language.
Customers view their payment history and request refunds.
Alternate flow: Customers input invalid email address which results in an error.
Pre-Conditions: Customers have a valid email address and device that can access the internet.
Post-Conditions: Customers have an account that they can use to navigate the website.

3.3.2 Use Case #2

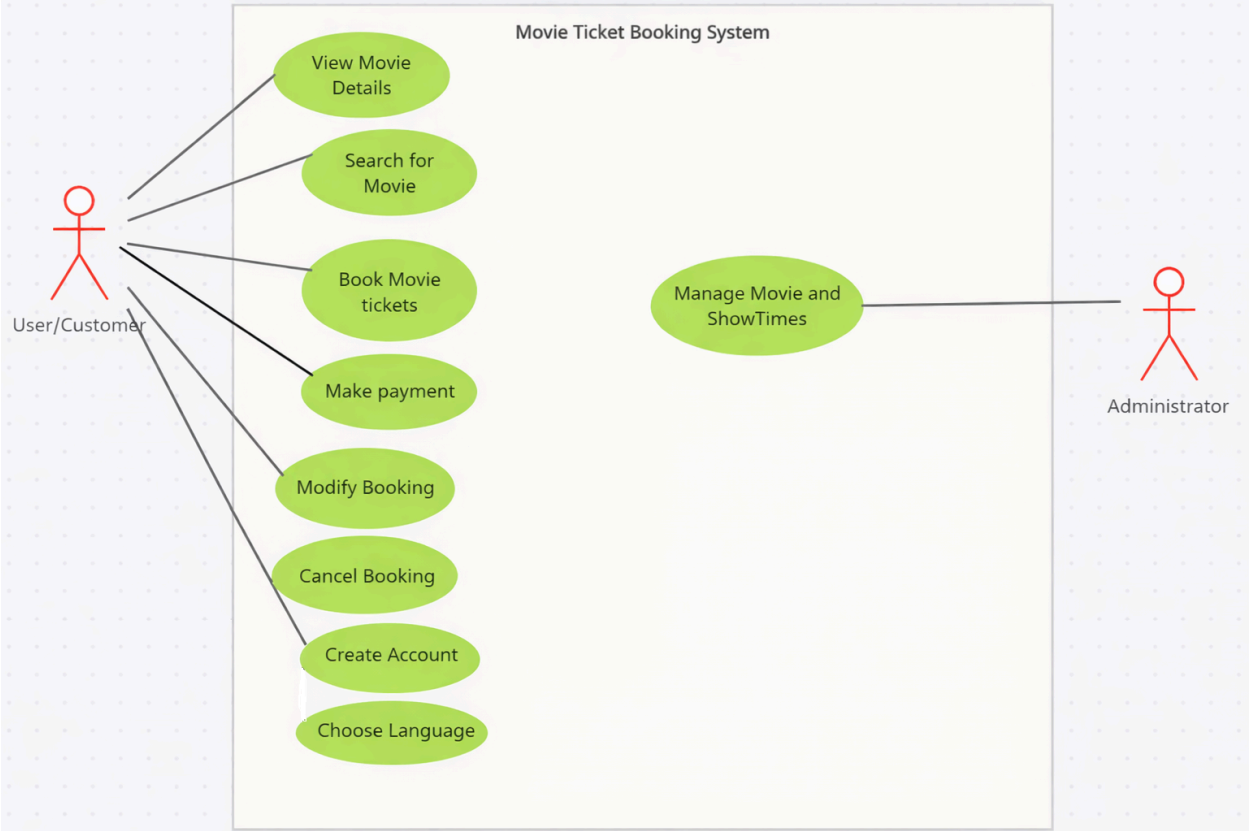
Name: Purchase a movie Ticket and optional concessions
Description: The customer is able to find a movie and view available options for purchasing a ticket as well as an option to purchase concessions.
Actors: Customer
Basic flow: The customer searches for a movie they are interested in.
Customer sees information about the film
Customer clicks a button to view locations and times
Customer chooses and option and chooses a seat on the seat map
Customer clicks to purchase a ticket and chooses a payment method
Customer chooses if they want to order concessions
Customer enters their information and receives an email confirmation after purchasing
Alternate flow: The movie is not available because the movie is not in theatres or there is no availability for the movie
Pre-Conditions: The user has an account and finds a movie showing that they want to attend
Post-Conditions: The user has a ticket QR code that is sent to them through their email.

3.3.3 Use Case #3

Name: Website management
Description: Administrators can change log into an editing view where they can make relevant changes to showings
Actors: Administrators
Basic flow: Admin logs in to editing mode
Admin updates movie information
Admin adds movies
Admin removes movies
Admin edits available locations
Admin edits available times
Admin edits available seats
Admin edits pricing

Alternate flow: Admin receives an error notifying them that an information box is missing an input
Pre-Conditions: Admin is able to log into an editing view.
Post-Conditions: Information regarding available movies, movie information, available locations, available times, available seats, and price is all updated

Use Case Model



3.4 Classes / Objects

3.4.1 <Class / Object #1>

User

3.4.1.1 Attributes
Username:String
Password:String
Email:String

3.4.1.2 Functions
registerAccount(username:String, password:String, email:String) : User
login(username:String, password:String) : boolean
updateEmail(newEmail:String) : boolean
logout() : void

<Reference to functional requirements and/or use cases>
(Use Case 1) Allows customers to create an account, purchase tickets, and view payment history.

3.4.2 <Class / Object #2>

Movie

3.4.2.1 Attributes
movieID : string
Title : String
Description : String
Genre : String
Duration : Int
Release date : Date
Availability : Boolean

3.4.2.2 Functions
movieDetails() : Void
getAvailability(status:Boolean) : void

<Reference to functional requirements and/or use cases>
(Use Case 2) Allows users to search for a movie, see the details of the movie, and view the availability of the film.

3.4.2 <Class / Object #3>

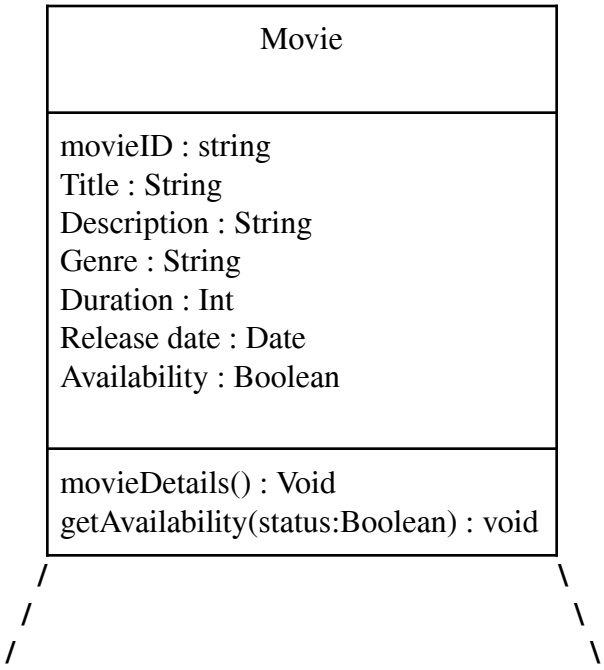
Management

3.4.3.1 Attributes
adminID : String
name : String
Email : String
Role : String

3.4.3.2 Functions
addMovie(movie:Movie) : void
removeMovie(movieID:String) : void
editMovie(movieID:String) : void
manageAvailability(movieID:String, status:Boolean) : void

<Reference to functional requirements and/or use cases>
(Use Case 3) Allows administrators to log into the editing view and update or add to the website.

Class Diagrams



User	Management
Username:String Password:String Email:String	adminID : String name : String Email : String Role : String
registerAccount(username:String, password:String, email:String) : User login(username:String, password:String) : boolean updateEmail(newEmail:String) : boolean logout() : void	addMovie(movie:Movie) : void removeMovie(movieID:String) : void editMovie(movieID:String) : void manageAvailability(movieID:String, status:Boolean) : void

3.5 Non-Functional Requirements

3.5.1 Performance

The system should be built with speed and efficiency in mind. Searching for a movie's availability should take < 2 seconds. A display of available times and locations should appear in

<3 seconds. The seat map should be displayed in < 1 second. Numbers of concurrent users up to 500 should be supported without arising problems if the number is passed.

3.5.2 Reliability

Data should be backed up every day to avoid loss of anything important. Listings and availability should be updated as needed at all times to avoid irrelevant information. Showing times, seats, and locations should be consistently updated.

3.5.3 Availability

The service should be available in more than one place. A familiar system should be available on the desktop, mobile, and in-theatre kiosks. The site should aim to be up for 99% of the time during the year.

3.5.4 Security

User information and data should be the main focus when it comes to security. Credit card information in the database should be protected by account passwords and email confirmations. Passwords should be hashed and salted in the database. Multifactor authentication should be implemented for administrative use.

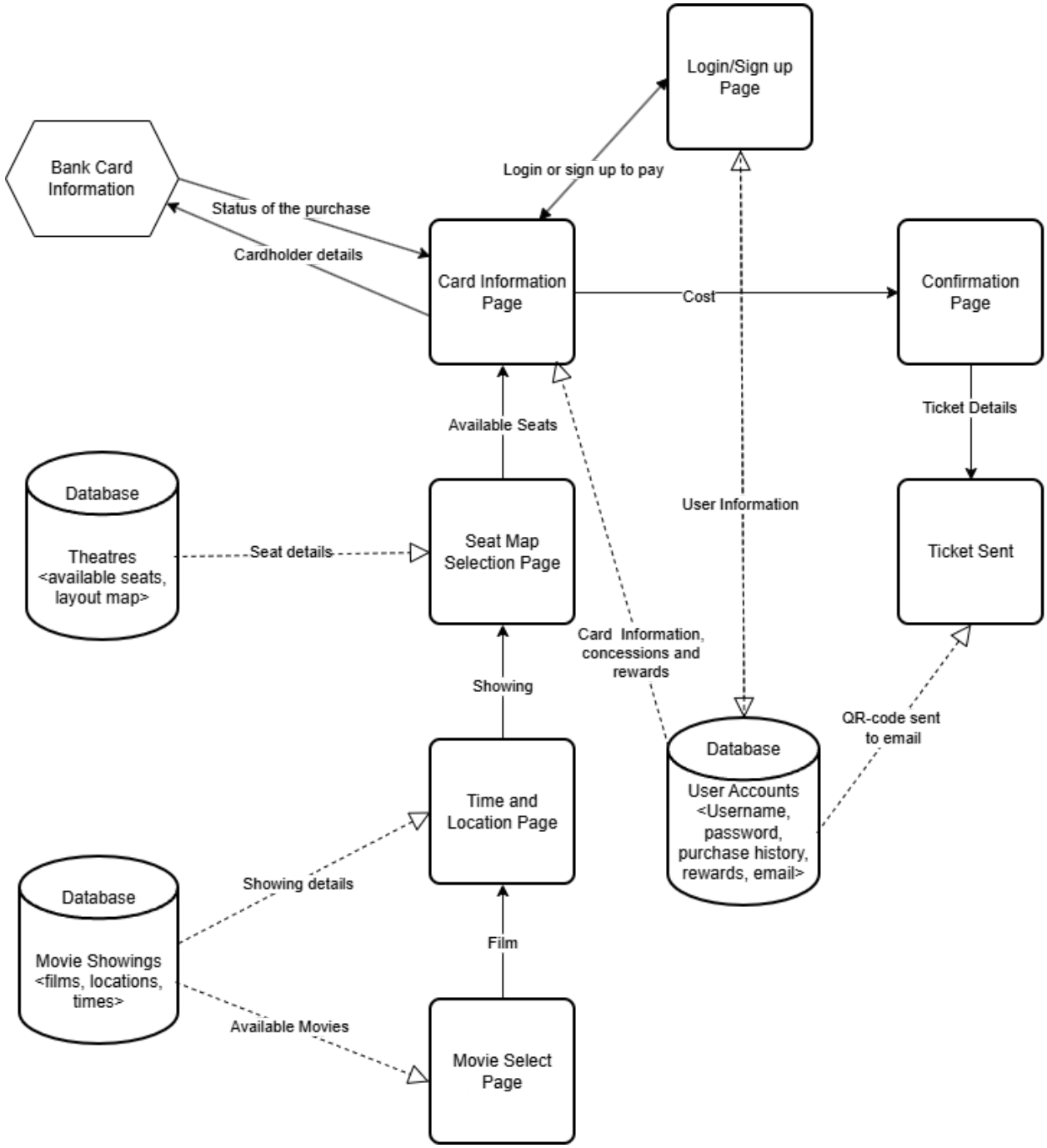
3.5.5 Maintainability

The system should be built with future updates or changes in mind. Code should be modular, and documentation should be clear for readability.

3.5.6 Portability

The system should be able to run on a web browser (Chrome, Edge) as well as on mobile. Microsoft Windows operating systems should be supported. Multiple different specs and systems should be supported.

SWA Diagram:


Login/Sign up Page
Bank Card Information
Login or sign up to pay
Status of the purchase
Cardholder details
Card Information Page
Cost
Confirmation Page
Ticket Details
Available Seats
Database
Theatres <available seats, layout map>
Seat details
Seat Map Selection Page
User Information
Ticket Sent
Card Information, concessions and rewards
Showing
QR-code sent to email
Database
User Accounts <Username, password, purchase history, rewards, email>
Time and Location Page
Database
Movie Showings <films, locations, times>
Showing details
Film
Available Movies
Movie Select Page

SWA Description:

Components:

I. **Movie Select Page:** This allows the user to browse all available films. This page retrieves all of the available movies from the movie showings database, as well as sends a film to the time and location page for the viewer to select.

II. **Time and Location:** This page grants the user the ability to choose a time and theater location. It pulls details from the Movie Showing database and passes the data to the seat map selection page to update.

III. **Seat Map Selection:** Shows an interactive layout of a theater for a chosen location and movie showing. Receives data from the Theaters database and more data from the time and location page. It passes available seats to card information and receives data from the user accounts database.

IV. **Card Information:** This is where the user enters their card information and cardholder details and seat selection. It sends a status of purchase to card information and prompts a user to login or sign up to pay for their tickets. This then passes the cost to the confirmation page.

V. **Login and Sign Up Page:** This page allows users to authenticate their information or to sign up for an account. This feeds user information to the seat map selection. This also allows for unique customizable rewards, concessions, and ticket data.

VI. **Confirmation Page:** This allows users to see a summary of their ticket and concessions as well as rewards. Receives cost and ticket details.

VII. **Ticket Sent:** This shows when the ticket is delivered and sends a QR code to the user's email. Data is taken from the User Accounts database.

Databases:

I. **Theaters:** This stores available tickets, seats, and the layout map. It sends seat details to the map selection page.

II. **Movie Showings:** This stores films, locations and titles. It sends Available Movies to Movie Select Page and Details to the Time and Location Page

III. **User Accounts:** This stores username, password, purchase history, rewards, and email. It sends a QR code to the email stored once the ticket is purchased. It also sends saved card information and rewards.

High Level Flow Connections:

I. **Movie Select -> Time and Location:** The user selects a film and the film is passed forward to the time and location page.

II. **Time and Location -> Seat Map Selection:** A specific showing is selected by the user, the time and location is passed forward to the seat map selection page.

III. **Seat Map Selection -> Card Information:** The user chooses a seat or multiple seats from the seat map which is passed forward to the card information page.

IV. **Card Information <--> Login/Sign up:** The user must login or sign up before entering card information.

V. **Card Information -> Confirmation:** The user enters their card information which is saved and passed forward to the confirmation page.

VI. **Confirmation -> Ticket Sent:** Purchase is confirmed and ticket information is passed forward to the user's email.

Database Connections:

I. **Movie Showings Database -> Movie Select:** The list of available movies is retrieved from the movie showings database.

II. **Movie Showings Database -> Time and Location:** The times and locations of featured showings are retrieved from the movie showings database.

III. **Theatres Database -> Seat Map Selection:** The map of available seats is retrieved from the theatres database.

IV. **User Accounts Database -> Card Information:** Card details, reward availability and optional concessions are retrieved from the user accounts database.

V. **User Accounts Database -> Ticket Sent:** User email is retrieved from the user account database to send the ticket QR-code.

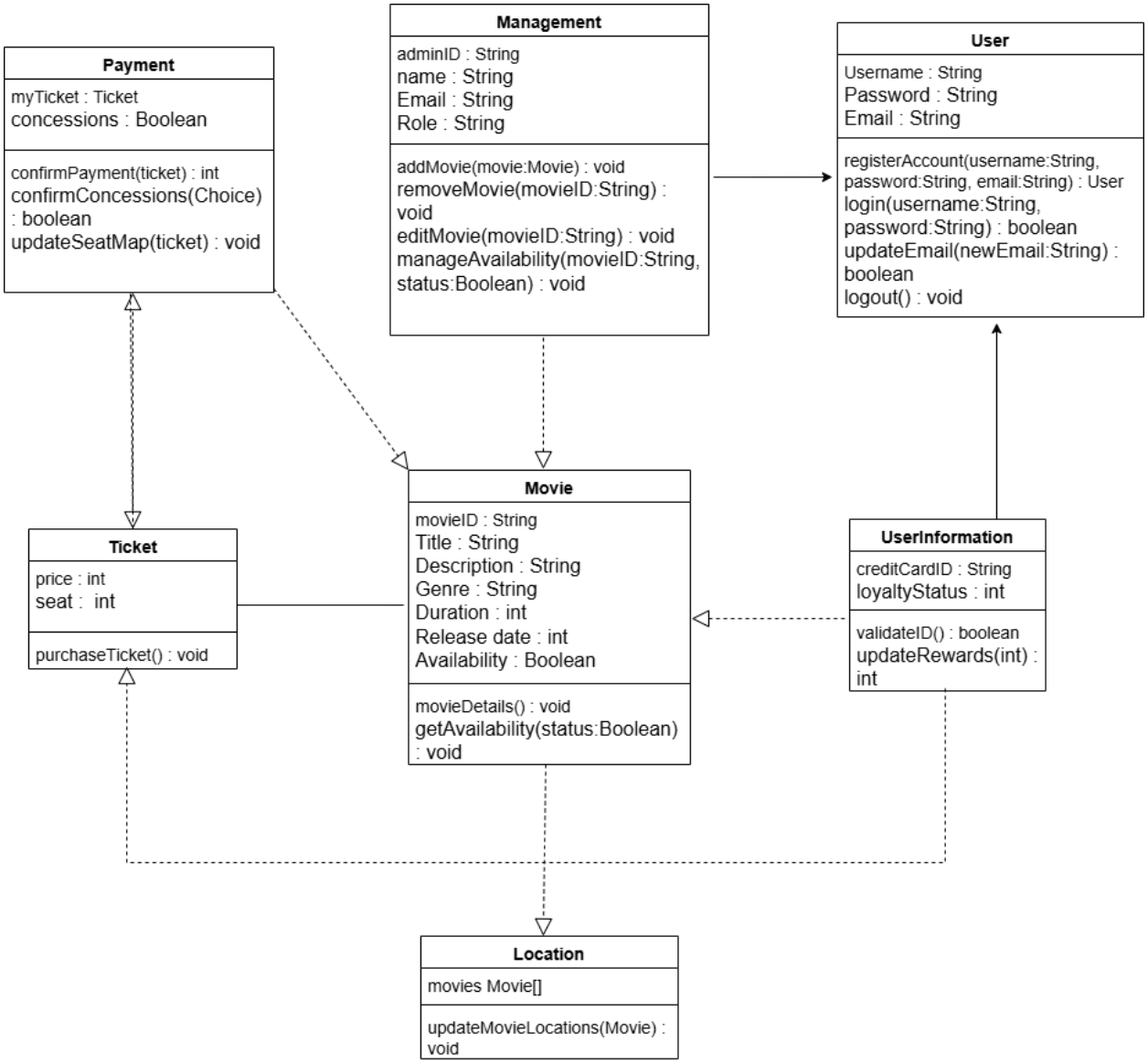
VI. **User Accounts Database <- Login/Sign up:** User emails, usernames and passwords are stored in the user accounts database.

Bank Card Information Connections:

I. **Bank Card Information -> Card Information:** The status of the purchase (approved/declined) is received from the bank card payment system.

II. **Bank Card Information <- Card Information:** The cardholder details and amount due are sent to the payment system.

UML Diagram:


Management
adminID : String
name : String
Email : String
Role : String
addMovie(movie:Movie) : void
removeMovie(movieID:String) : void
editMovie(movieID:String) : void
manageAvailability(movieID:String, status:Boolean) : void
User
Username : String
Password : String
Email : String
registerAccount(username:String, password:String, email:String) : User
login(username:String, password:String) : boolean
updateEmail(newEmail:String) : boolean
logout() : void
Payment
myTicket : Ticket
concessions : Boolean
confirmPayment(ticket) : int
confirmConcessions(Choice) : boolean
updateSeatMap(ticket) : void
Movie
movieID : String
Title : String
Description : String
Genre : String
Duration : int
Release date : int
Availability : Boolean
movieDetails() : void
getAvailability(status:Boolean) : void
Ticket
price : int
seat : int
purchaseTicket() : void
UserInformation
creditCardID : String
loyaltyStatus : int
validateID() : boolean
updateRewards(int) : int
Location
movies Movie[]
updateMovieLocations(Movie) : void

UML Description:

The UML Diagram consists of 7 classes: **Management, User, Movie, Ticket, Payment, UserInformation, and Location.**

I. **Management:**
The **Management** class represents a staff member who can manage Movie class information and information in the User class.

Attributes:
- **adminID : String** - This holds a unique admin ID for each staff member who is authorized to use and access backend operations.
- **name : String** - This holds the staff member's name.
- **email : String** - This holds the staff member's email address.
- **role : String** - This holds the level of access for each staff member (i.e. admin, staff, etc.)

Operations:
- **addMovie(movie:Movie) : void** - This creates and adds a new movie into the system and into a specific location.
- **removeMovie(movieID:String) : void** - This removes a movie from the system of a location.
- **editMovie(movieID:String) : void** - This function edits the details of a created movie.
- **manageAvailability(movieID:String, status:Boolean) : void** - This function toggles the availability of a movie

II. **User:**
The **User** class represents a registered patron of any one of the movie chains involved in the system.

Attributes:
- **Username : String** - This holds the chosen username for the patron.
- **Password : String** - This holds the patron's chosen password.
- **Email : String** - This holds the patron's email address.

Operations:
- **registerAccount(username:String, password:String, email:String) : User** - This operation creates a new account for a patron.
- **login(username:String, password:String) : boolean** - This operation allows for a user to log into their unique account and grants access to their user information.
- **updateEmail(newEmail:String) : boolean** - This allows for a patron to update their email address.
- **logout() : void** - This logs out a user from their account

III. Movie:

The **Movie** class represents a movie in the system, which can be updated by Management.

Attributes:

- **movieID : String** - This holds a unique identifier for a single movie.
- **Title : String** - This holds a movie's title.
- **Description : String** - This holds a description of the movie.
- **Genre : String** - This holds the genre tags for a movie.
- **Duration : int** - This holds the run time of a movie in minutes.
- **Release date : int** - This holds the release date of a specific movie.
- **Availability : Boolean** - This holds a value that shows whether the movie is currently being shown in a location or not.

Operations:

- **movieDetails() : void** - This displays the details of a specific movie.
- **getAvailability(status:Boolean) : void** - This shows the availability of a certain movie.

IV. Ticket:

The **Ticket** class represents a purchased ticket from a user. It allows for access and entrance into a theatre.

Attributes:

- **price : int** - This holds the price for a ticket.
- **seat : int** - This holds the seat number for a purchased ticket.

Operations:

- **purchaseTicket() : void** - This operation allows the user to purchase a ticket.

V. Payment:

The **Payment** class handles the transaction and purchase of Tickets and communicates with the payment portal.

Attributes:

- **myTicket : Ticket** - This holds the ticket that the patron has selected for payment.
- **concessions : Boolean** - Holds true or false whether concessions are purchased or not.

Operations:

- **confirmPayment(ticket) : int** - Confirms payment has been processed and returns confirmation code.
- **confirmConcessions(Choice) : boolean** - Confirms payment has been processed and returns confirmation code.

- **updateSeatMap(ticket) : void** - Updates seating map, making purchased tickets unavailable for further purchase.

VI. UserInformation:

The **UserInformation** class allows users to save and store their information for more convenient purchasing, as well as store their loyalty rewards and status.

Attributes:

- **creditCardID : String** - This holds the patron's saved payment information.
- **loyaltyStatus : int** - This holds the users current loyalty status and points

Operations:

- **validateID() : boolean** - Confirms user's identity and information.
- **updateRewards(int) : int** - This updates loyalty status and rewards after a purchase. It also returns an updated count of the user's points.

VII. Location;

The **Location** class represents a location of a theater that is under the system.

Attributes:

- **movies : Movie[]** - This displays all the movies available at a specific location.

Operations:

- **updateMovieLocations(Movie) : void** - This allows for Management to update the movies at a specific location.

Overview of Software System

The movie ticketing system is a web based application made for a theater company to manage online ticket sales and movie information. The application is made to make the ticket purchasing process simple, efficient, and convenient for all users, supporting multiple languages. Customers can search up and browse through a database of movies, viewing movie details, showtimes, and theater locations. Customers are able to pick a movie, a theater, and then look through available seats on a seat map. After choosing their preferred options, customers can purchase tickets and preorder concessions online.

Customers can create exclusive theater accounts that store purchase history and loyalty points received from past purchases. They can view previous purchases and request refunds if necessary and meet requirements. Upon purchase, tickets are generated with a unique QR code, sent to the customer's email with confirmation of their tickets. These tickets can be stored in mobile wallets for ease of use.

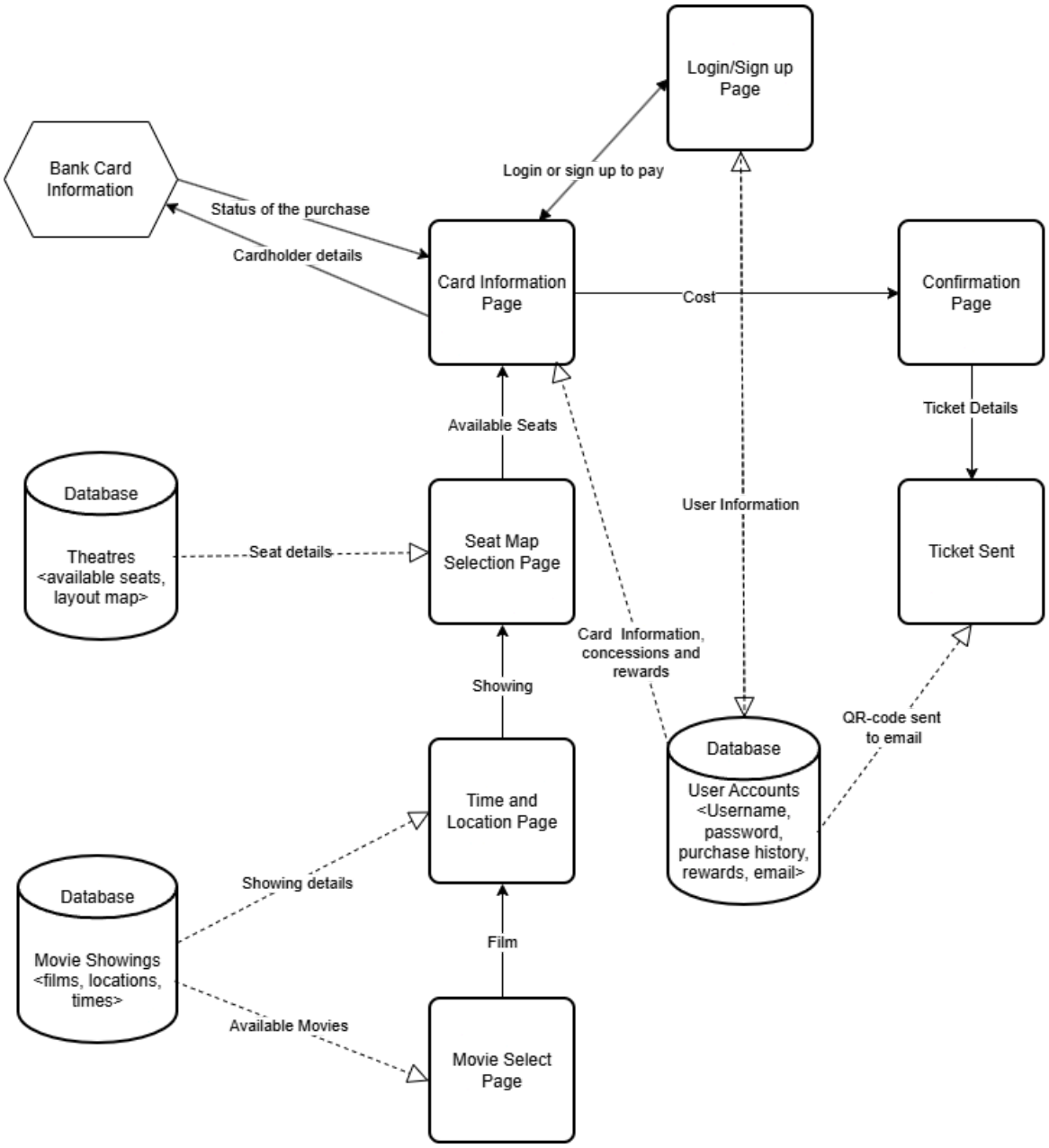
Payments for tickets and concessions are processed through external payment systems, which integrate with customers' email for confirmation details. Administrators of the application can log into a management interface to add, remove, or edit movies, update showtimes, locations, seat availability, and pricing. The system alerts admins if required information is missing.

The system is accessible through web browsers on desktop and mobile devices. It is designed to handle multiple concurrent users, manage high-demand showtimes with a queue, and limit the number of tickets per purchase.

Development Plan and Timeline:

Item to Implement	Team Member Responsible	Estimated Date of Completion
Movie Select Page	Jonathan	3/26/26
Time and Location Page	Jonathan	3/29/26
Seat Map Selection Page	Ian	3/31/26
Card Information Page	Ian	4/16/26
Login/sign up Page	Yosef	4/19/26
Confirmation page	Yosef	4/24/26
Ticket Sent Page	Yosef	4/30/26

Test Plan: Each part of the diagram will be tested to ensure the system works as intended. The different parts will need to be tested at the functional level, unit level and system level. The ability to search for a movie, choose a listing, earn rewards and purchase a ticket needs to be tested so that none of the databases fail to function and attain the data they need. Specific classes and functions need to be tested with relevant arguments to avoid failures in the code. The full processes of the system need to be tested to confirm that all actions and external services work for the user. Testing the user functions, code and system flow of the website will allow success at the functional level, unit level, and system level.



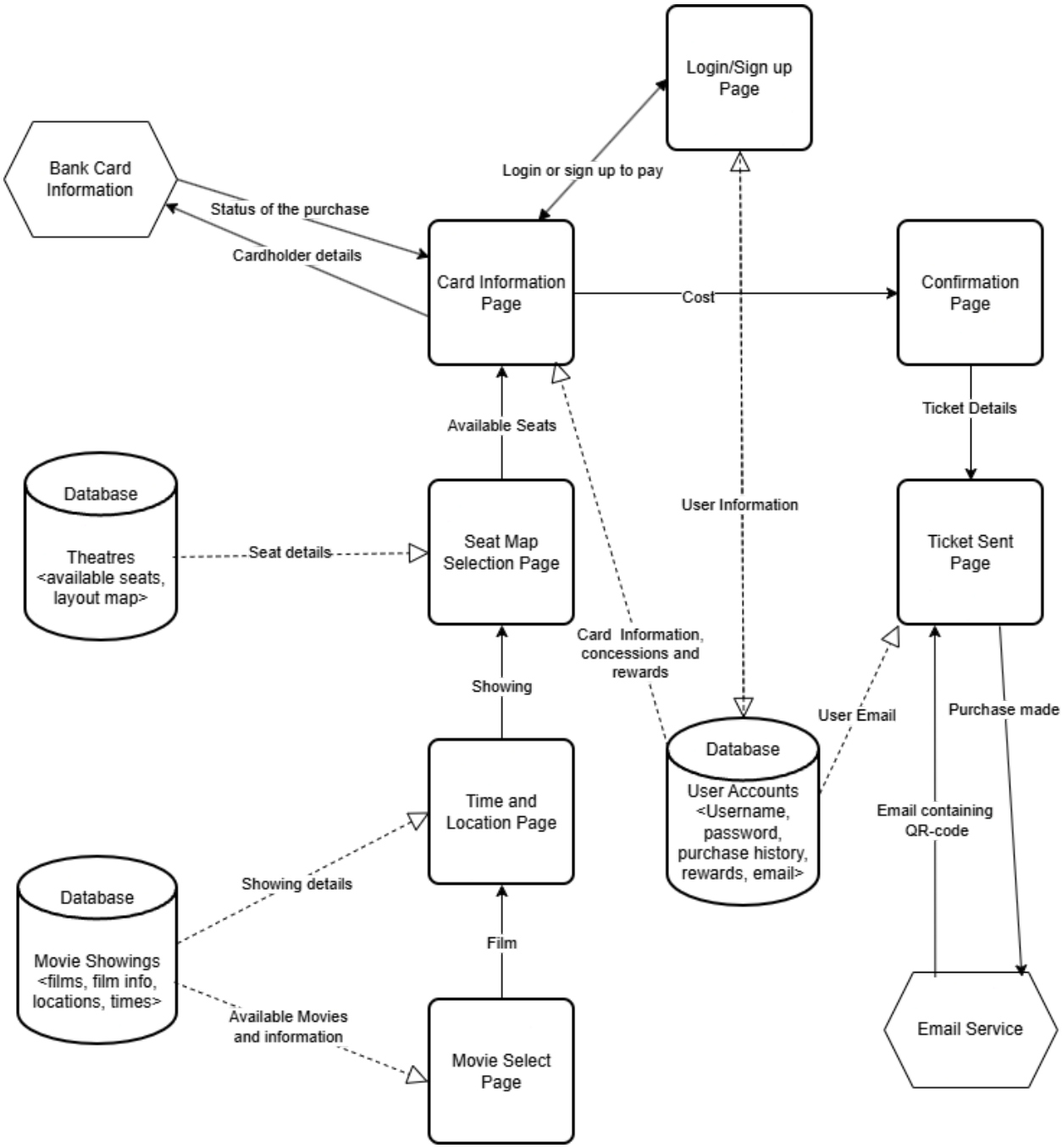
GitHub Repository Test Plan Links:

Ian Gillespie
https://github.com/igillespie1075/Movie-Ticketing-System---Group-14/blob/main/MovieTicketingSystem%20-%20Test%20Plan.pdf

Jonathan Clemente
https://github.com/igillespie1075/Movie-Ticketing-System---Group-14

Yosef Pilip
https://github.com/igillespie1075/Movie-Ticketing-System---Group-14

Updated Architecture:



Data Management Strategy:

The system will have persistent and sensitive data that can fit into different categories. So all data will need to be structured so that data can be managed efficiently. SQL databases will be used to achieve this since they allow us to easily manage multiple types of data such as films, locations, times, available seats, passwords, emails etc. We will have three separate databases that store data related to movie information, theatre information and user information. This keeps sensitive

data separate from persistent data in case of database breach. This will make querying of data more complex, but it will also allow our system to have more protection.This separation also clearly defines what data needs to be managed whether that's persistent data such as times and seat availability or sensitive data like hashed passwords.

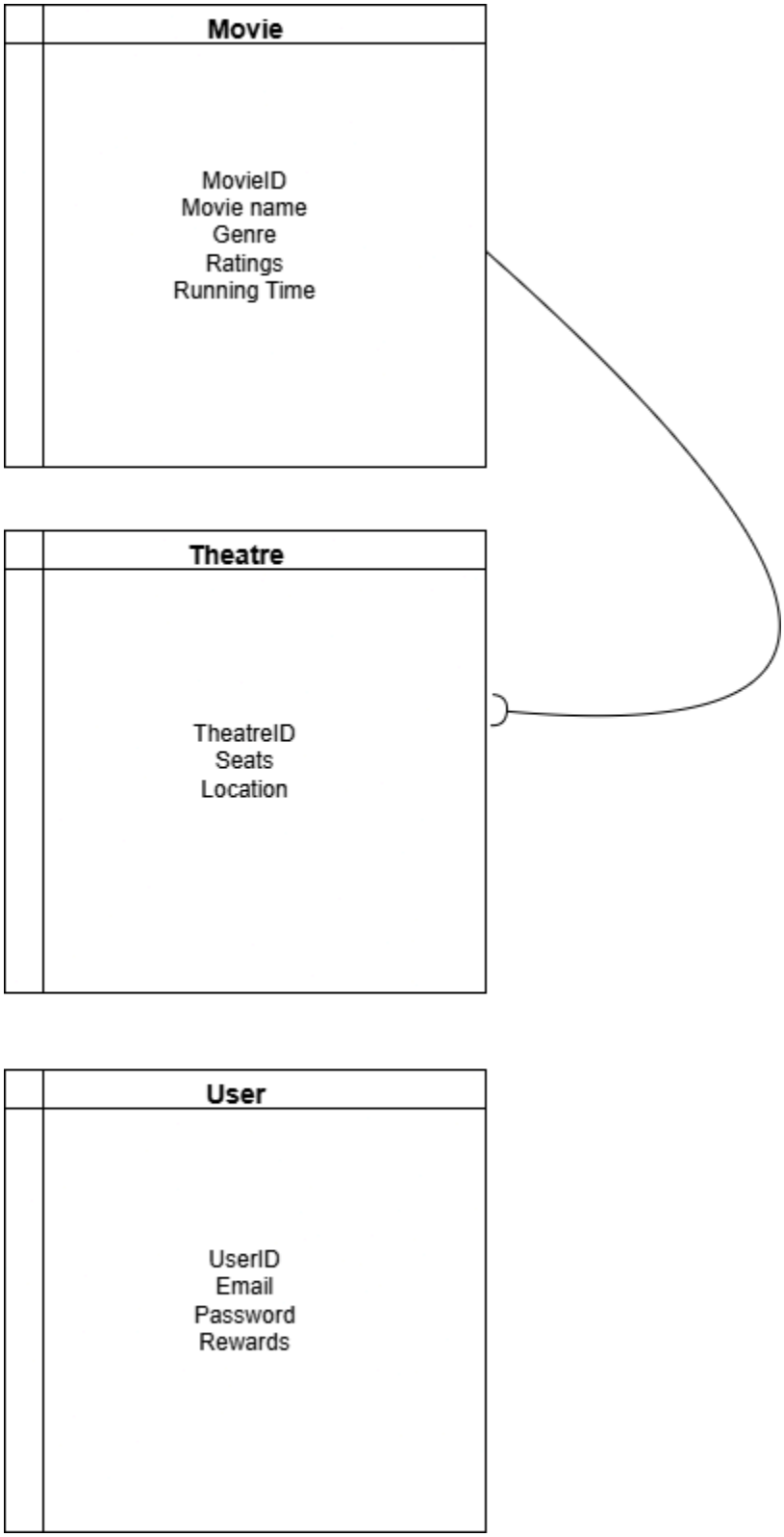
Examples of structured data:

MovieID	Movie name	Ratings	Genre	Running Time
3	name1	2	horror	75 minutes
6	name2	4	action	93 minutes

TheatreID	Seats	Location
6	seatMap	Grossmont center
3	seatMap	Rancho San Diego

UserID	Email	Password	Rewards
7	example@email.com	hashed(******)	13$
1	example2@email.com	hashed(********)	16$

Entity Relationship:



Data definitions

MovieID: Number used to identify a specific film
Movie Name: Title of the film
Genre: Type of film
Ratings: The quality of the film represented through scores
Running Time: The duration of the movie
TheatreID: Number used to identify a specific movie theatre
Seats: The available seats in a theatre
Location: The place where the theatre is located
UserID: Number used to identify a specific user account
Email: Online mail account
Password: The code used by users to login to their account
Rewards: Money allotted to users based on purchase history

Alternatives:

An alternative would be to have a simple list to store our data, having one large flat table containing all the information. However, this would be a problem because a list can contain more than one concept per row, which would create a mix of all our information, like our theaters, showtimes, and user data. This would create data anomalies where, when something is updated, it must be updated across every single row. If bookings were deleted, the information showing might also be deleted, creating deletion anomalies.

Another alternative would be to use a personal database system like Microsoft Access. While the setup is simpler, personal databases are set up for one user, which would not work for our ticketing system.

Database Logic, Structure, and Technology:

The data in this system is split up into three distinct databases: Movie Showings, Theatre, and User Accounts. We chose to separate our data this way in order to keep sensitive data safe in case of a breach of security, i.e., User Credentials remain safe if Movie Showings or Theatre are breached. Another reason why the databases are deliberately split up this way is in order to have consistency and easy access to each domain. This is because each database has very explicit and clear scopes that allow for consistent access. However, the tradeoff is that any query in this system is inherently more complex due to the number of databases required.

In regard to how the data is split up logically, the Movie Showings Database deals with any operation that requires individual films, their metadata (genre, rating, run time, location, showtimes, etc), and any queries that require it. The Theatre Database works on the same principle, but rather than working on films, it works on individual physical theatre locations. It uses theatre ID, seat maps, and location information. Finally, User Accounts handles anything with users, such as usernames and hashed passwords, email addresses, purchase history, and rewards.

Our system also uses SQL relational databases to manage multiple data types and allow for efficient and explicit access to data.